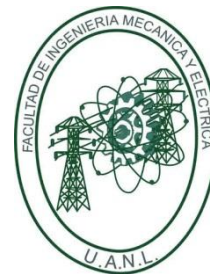




Universidad Autónoma de Nuevo León
Facultad de Ingeniería Mecánica y Eléctrica



Controladores y Microcontroladores
Programables

AFU01 - Examen de Medio Curso – Práctico

N.º de Equipo: 4

Brigada: 001

Nombre:	Matricula:	Numero de lista:
Heidi Pamela Martínez Martínez	1952947	6
Steven Neftalí Verdugo Figueroa	2132366	60
Ángel Armando Solís Jiménez	2132088	55
Carlos Emiliano Mendoza Ramírez	2048186	35

Nombre del profesor: Jesús Daniel Garza Camarena

Semestre Agosto – Diciembre 2025

Días de la clase y Hora: JueM4M6

San Nicolás de los Garza, N.L.

Fecha de entrega: 09/10/2025

REDACCIÓN DE LA ACTIVIDAD

Se propone el diseño de un sistema electrónico basado en un microcontrolador para controlar un motor de corriente directa con su controlador L293D, 4 LEDs y un display de 7 segmentos. Cada combinación de entrada activa una acción específica. La velocidad de las secuencias de los leds será elegida por el diseñador, los cambios en las salidas entre combinaciones deben de ser inmediatos.

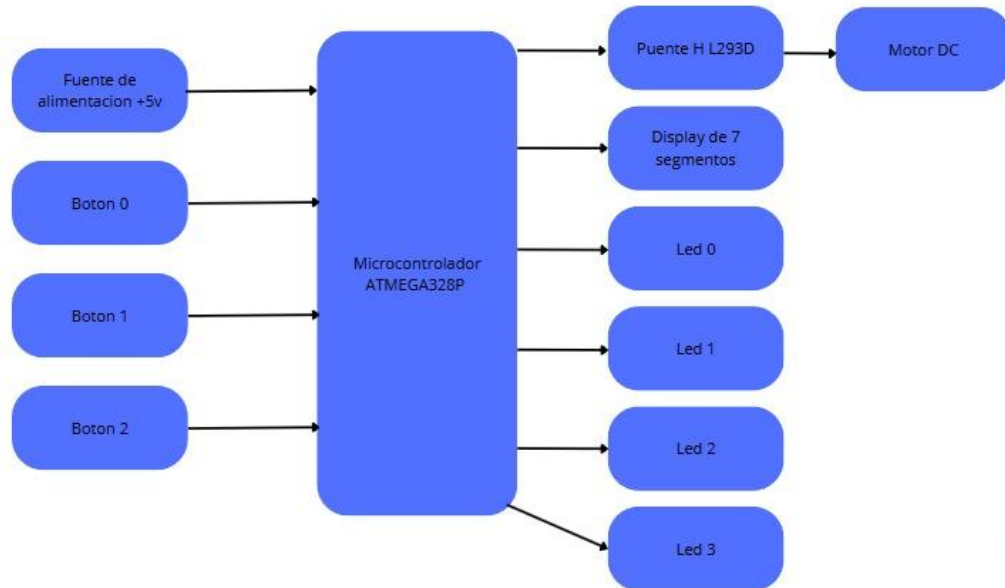
Las secuencias para implementar son las siguientes:

Valor de las Entradas	Acción del Motor	Acción del Display	Estado de los LEDs repetitivos
000	Motor apagado	Muestra "E"	Los LEDs del medio se encienden, luego los de las orillas
001	Gira hacia la izquierda	Muestra "1"	Un solo LED se desplaza de izquierda a derecha
010	Gira hacia la derecha	Muestra "2"	Un solo LED se desplaza de derecha a izquierda
011	Motor en modo "free-wheel"	Muestra "P"	Todos los LEDs parpadean
1XX	Motor en modo "fast-stop"	Muestra "F"	Todos los LEDs están completamente encendidos

X = don't care o no importa

AFU01 - Examen de Medio Curso - Práctico

1. DIAGRAMA DE BLOQUES



2. LISTA DE MATERIALES

Material	Marca	Precio (\$)	Unidades	Detalles técnicos
Arduino R3	STEREN	180	1	Microcontrolador ATmega328P; Voltaje de operación 5 V; Voltaje de entrada recomendado 7–12 V; Frecuencia del cristal 16 MHz; Memoria Flash 32 KB; 14 pines digitales (6 PWM), 6 pines analógicos.
Protoboard	STEREN	120	1	Placa de pruebas sin soldadura; compatible con cables tipo jumper; distribución estándar de buses de alimentación.
Resistencia	STEREN	10	15	Valor típico 220 Ω – 10 k Ω (según uso); tolerancia $\pm 5\%$; potencia $\frac{1}{4}$ W. Usadas para limitar corriente en LEDs y display.
Led	STEREN	3	4	Voltaje de operación 2 V – 3.3 V; corriente recomendada 20 mA; diámetro estándar 5 mm.
Jumpers	STEREN	10	19	Cables flexibles de 20 cm; tipo Dupont; aptos para conexiones en protoboard.
Display 7 segmentos	STEREN	15	1	Modelo similar a 5161AS; 8 pines de segmento (a–g + DP); voltaje típico de segmento 2 V; corriente

				10–15 mA por segmento; conexión a GND común.
Botones	STEREN	2	3	Tipo momentáneo; dimensiones 6×6 mm; tensión máxima 12 VDC; corriente máxima 50 mA.
L293D	STEREN	19	1	Controlador dual de motor DC; voltaje de alimentación 4.5–36 V; corriente de salida 600 mA por canal; diodos internos de protección.
Motor CD	STEREN	49	1	Voltaje de operación 3–9 VDC; velocidad aprox. 6000 RPM a 6 V; corriente nominal 250 mA; polaridad reversible.
Batería 9V	STEREN	179	1	Salida 9 V.

3. FUNDAMENTO DEL COMPORTAMIENTO

1. Especificaciones del Sistema

Entradas:

- BTN0 (PC2) - Botón de control 0
- BTN1 (PC3) - Botón de control 1
- BTN2 (PC6) - Botón de función especial

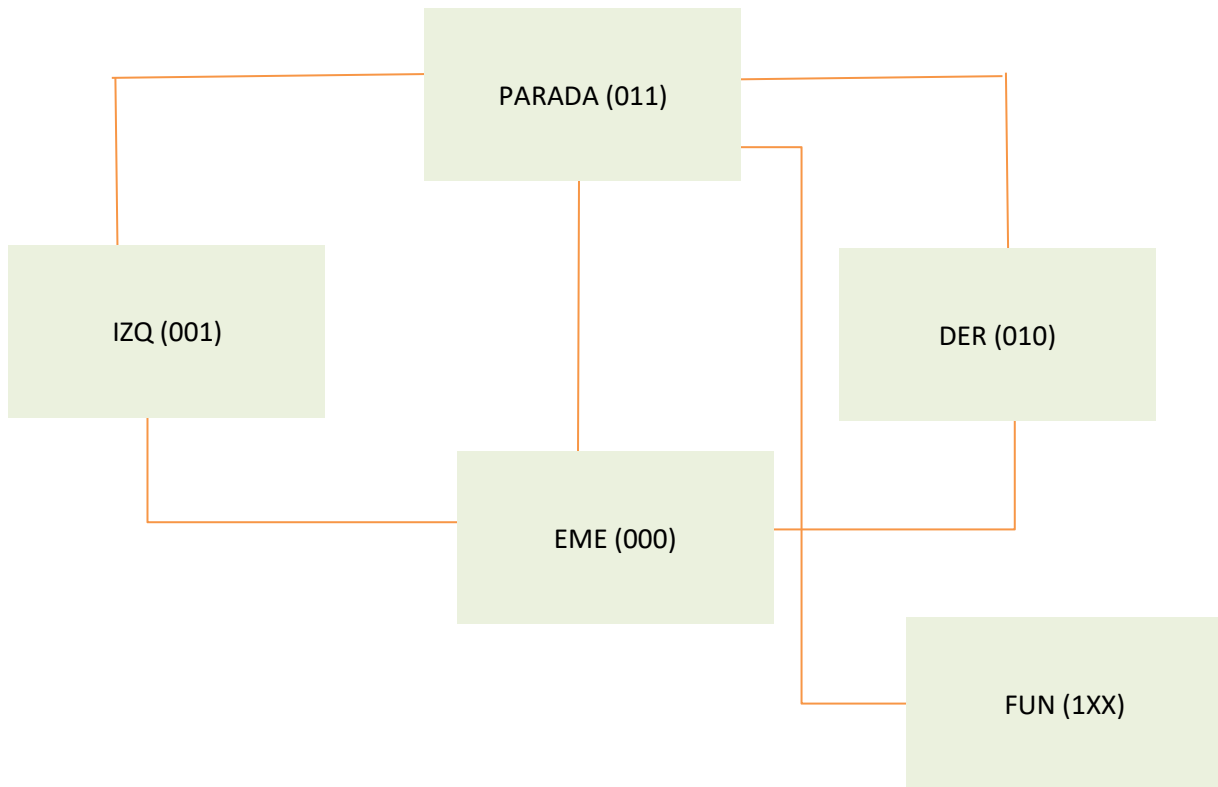
Salidas:

- Motor DC (L293D): IN1, IN2, EN1
- Display 7-segmentos: Segmentos A-G
- LEDs indicadores: LED0, LED1, LED2, LED3

2. Tabla de Verdad

BTN2	BTN0	BTN1	Estado (nombre)	Display (carácter)	Efecto LEDs	Motor: EN1	Motor: IN1	Motor: IN2
					center → edges (one- shot) shift			
0	0	0	STOP (E)	'E'		0	0	0
0	0	1	LEFT (1)	'1'	left→right (one- shot) shift	1	0	1
0	1	0	RIGHT (2)	'2'	right→left (one- shot)	1	1	0
0	1	1	FREE (P)	'P'	blink all	0	0	0
1	X	X	STOP (F)	'F'	All on	0	0	0

3. Máquina de Estados Finitos (FSM)



4. Tabla de Estados

Estado Actual	Entradas BTN1:BTN0	Estado Siguiente	Acciones
PARADA	00	EMERGENCIA	Display 'E', Fast Stop, LEDs Centro→Orillas
PARADA	01	IZQUIERDA	Display '1', Turn Left, LEDs ↔
PARADA	10	DERECHA	Display '2', Turn Right, LEDs →←
PARADA	11	PARADA	Display 'P', Free Wheel, LEDs Parpadeo

Estado Actual	Entradas BTN1:BTN0	Estado Siguiente	Acciones
IZQUIERDA	00	EMERGENCIA	Display 'E', Fast Stop, LEDs Centro→Orillas
IZQUIERDA	01	IZQUIERDA	Display '1', Turn Left, LEDs ←→
IZQUIERDA	10	DERECHA	Display '2', Turn Right, LEDs →←
IZQUIERDA	11	PARADA	Display 'P', Free Wheel, LEDs Parpadeo
DERECHA	00	EMERGENCIA	Display 'E', Fast Stop, LEDs Centro→Orillas
DERECHA	01	IZQUIERDA	Display '1', Turn Left, LEDs ←→
DERECHA	10	DERECHA	Display '2', Turn Right, LEDs →←
DERECHA	11	PARADA	Display 'P', Free Wheel, LEDs Parpadeo
EMERGENCIA	00	EMERGENCIA	Display 'E', Fast Stop, LEDs Centro→Orillas
EMERGENCIA	01	IZQUIERDA	Display '1', Turn Left, LEDs ←→
EMERGENCIA	10	DERECHA	Display '2', Turn Right, LEDs →←

Estado Actual	Entradas BTN1:BTN0	Estado Siguiete	Acciones
EMERGENCIA	11	PARADA	Display 'P', Free Wheel, LEDs Parpadeo

5. Ecuaciones Lógicas

Enable del Motor:

$EN1 = \text{NOT}(\text{BTN0 AND BTN1}) \text{ AND } (\text{BTN0 OR BTN1})$

Dirección del Motor:

$IN1 = \text{BTN0 AND NOT}(\text{BTN1})$ ----- Derecha

$IN2 = \text{NOT}(\text{BTN0}) \text{ AND } \text{BTN1}$ ----- Izquierda

6. Tablas de Verdad del Display 7-Segmentos

Carácter	Segmentos Encendidos	Binario (ABCDEFG)
1	B, C	0110000
2	A, B, G, E, D	1101101
E	A, F, G, E, D	1111001
P	A, B, G, F, E	1110011
F	A, F, G, E	1110001

7. Mapeo a Pines del Microcontrolador

Segmento	Pin Puerto	Bit
A	PORTB0	0
B	PORTB1	1

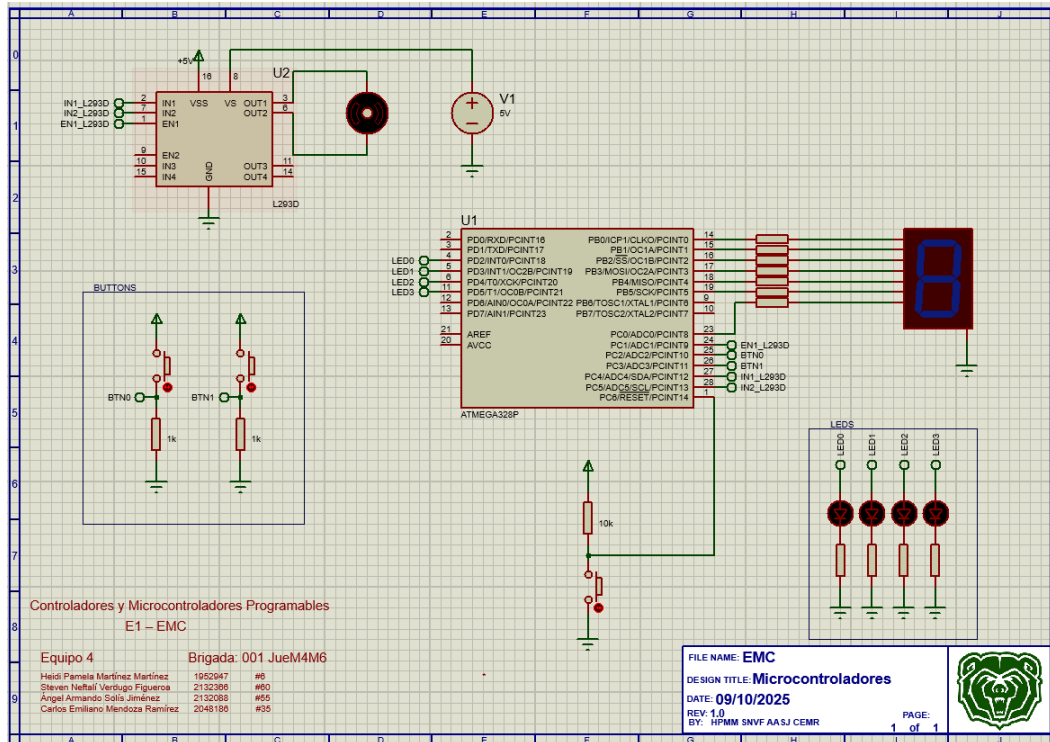
Segmento	Pin Puerto	Bit
C	PORTB2	2
D	PORTB3	3
E	PORTB4	4
F	PORTB5	5
G	PORTC0	0

8. Secuencias de LEDs

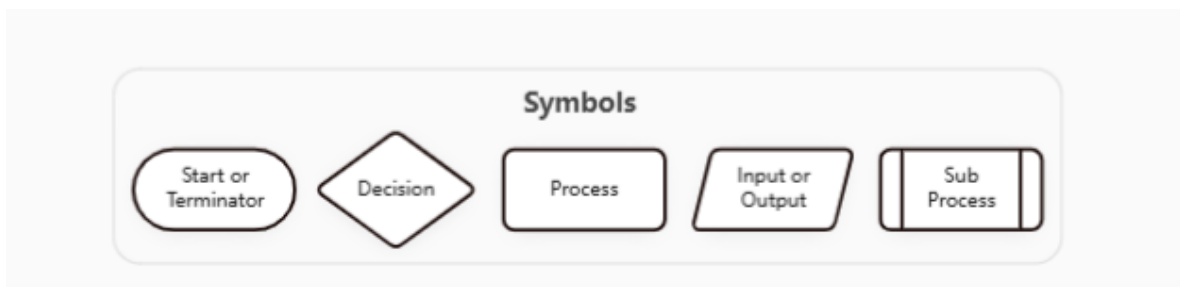
Función	Patrón LEDs	Secuencia
Centro→Orillas	0110 → 1001	LED1,2 → LED0,3
Izq→Der	1000 → 0100 → 0010 → 0001	LED0→LED1→LED2→LED3
Der→Izq	0001 → 0010 → 0100 → 1000	LED3→LED2→LED1→LED0
Parpadeo	1111 → 0000	Todos ON→OFF

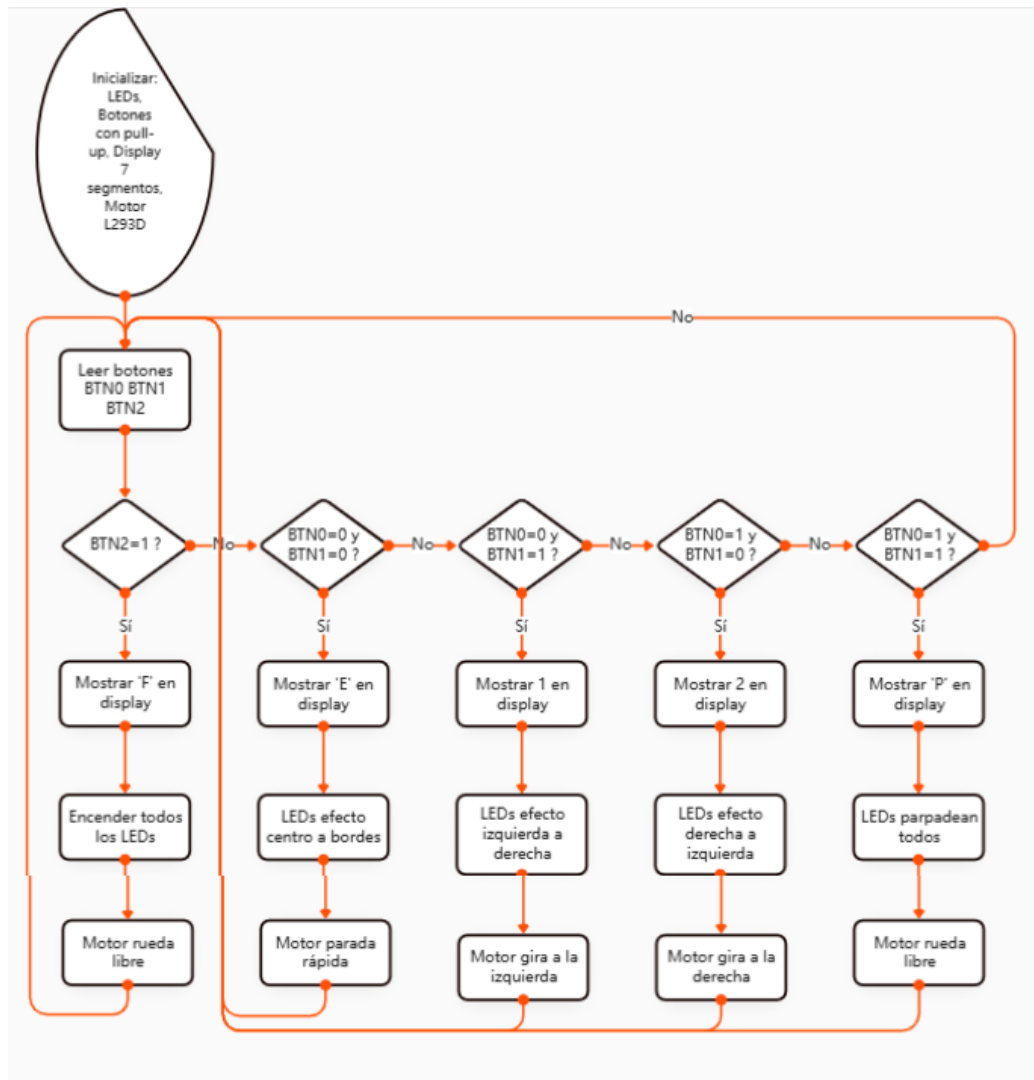
4. DIAGRAMA ESQUEMÁTICO EN EDA

ESQUEMÁTICO EN PROTEUS



5. DIAGRAMA DE FLUJO EN ALTO NIVEL





6. CÓDIGO EN LENGUAJE C o ASM NO ARDUINO

```

/*****
*   Copyright (C) 2025 by Equipo 4
*   Heidi Pamela Martínez Martínez      1952947    #6
*   Steven Neftalí Verdugo Figueroa    2132366    #60
*   Ángel Armando Solís Jiménez        2132088    #55
*   Carlos Emiliano Mendoza Ramírez    2048186    #35
*
*   Attribution-NonCommercial-ShareAlike 3.0(CC-BY-NC-SA 3.0)
*   AFU01 - Examen de Medio Curso - Práctico
*   Dispositivo: atmega328p
*   Lenguaje C
*   Rev: 1.0
*   Fecha: 09/10/25
*****/
#define F_CPU 16000000UL // 16 MHz - Frecuencia del cristal

//LIBRERIAS

```

```

#include <avr/io.h>
#include <util/delay.h>

#define RETARDO 200 // Tiempo base para delays en milisegundos

//LEDS - Configuración de pines para los 4 LEDs en PORTD
#define LEDS_DDRX DDRD
#define LEDS_PORTX PORTD
#define LEDS_LED0 PORTD2
#define LEDS_LED1 PORTD3
#define LEDS_LED2 PORTD4
#define LEDS_LED3 PORTD5

// Array para manejar fácilmente los LEDs en secuencias
uint8_t LEDS[4] = { LEDS_LED0, //0
    LEDS_LED1, //1
    LEDS_LED2, //2
    LEDS_LED3}; //3

//BOTONES - Configuración de pines para botones en PORTC
#define BUTTONS_DDRX DDRC
#define BUTTONS_PORTX PORTC
#define BUTTONS_PINX PINC
#define BUTTONS_BTN0 PINC2
#define BUTTONS_BTN1 PINC3
#define BUTTONS_BTN2 PINC6

//MACRO BOTONES - Lectura de estado de botones (0=presionado, 1=no presionado)
#define BUTTONS_BTN0_READ (!(BUTTONS_PINX & (1<<BUTTONS_BTN0)))
#define BUTTONS_BTN1_READ (!(BUTTONS_PINX & (1<<BUTTONS_BTN1)))
#define BUTTONS_BTN2_READ (!(BUTTONS_PINX & (1<<BUTTONS_BTN2)))

//DISPLAY ANODO COMUN - Configuración de pines para display 7 segmentos
#define DISPLAY_7SEG_AF_DDRX DDRB
#define DISPLAY_7SEG_AF_PORTX PORTB
#define DISPLAY_7SEG_G_DDRX DDRC
#define DISPLAY_7SEG_G_PORTX PORTC
#define DISPLAY_7SEG_SEGA PORTB0
#define DISPLAY_7SEG_SEGB PORTB1
#define DISPLAY_7SEG_SEGC PORTB2
#define DISPLAY_7SEG_SEGD PORTB3
#define DISPLAY_7SEG_SEGE PORTB4
#define DISPLAY_7SEG_SEGF PORTB5
#define DISPLAY_7SEG_SEGG PORTC0

//L293D - MOTOR DC - Configuración de pines para control del motor
#define L293D_DDRX DDRC
#define L293D_PORTX PORTC
#define L293D_IN1 PORTC5
#define L293D_IN2 PORTC4
#define L293D_EN1 PORTC1

//DECLARACIONES DE FUNCIONES
void leds_init(void);
void leds_blink_all(void);
void leds_center_to_edges(void);
void leds_shift_left_to_right(void);
void leds_shift_right_to_left(void);

```

```

void leds_all_on(void);
void leds_all_off(void);
void buttons_init(void);
void display_init(void);
void display_show_number(uint8_t number);
void l293d_init(void);
void l293d_motor_fast_stop(void);
void l293d_motor_free_wheel(void);
void l293d_motor_turn_right(void);
void l293d_motor_turn_left(void);

//// ----- MAIN -----
//-----
//-----

int main(void)
{
    //llamar funciones de inicialización
    leds_init();
    buttons_init();
    display_init();
    l293d_init();
    uint8_t i = 0;

    // Loop principal infinito
    while (1)
    {
        //00
        if (!(BUTTONS_BTN0_READ) && !(BUTTONS_BTN1_READ))
        {
            display_show_number('E'); // Mostrar 'E' en display
            leds_center_to_edges();   // Efecto de LEDs centro-orillas
            l293d_motor_fast_stop();  // Parada rápida del motor
        }

        //01
        if (!(BUTTONS_BTN0_READ) && (BUTTONS_BTN1_READ))
        {
            display_show_number(1);      // Mostrar '1' en display
            leds_shift_left_to_right();   // Efecto LEDs izquierda-
derecha
            l293d_motor_turn_left();      // Motor gira a la izquierda
        }

        //10
        if ((BUTTONS_BTN0_READ) && !(BUTTONS_BTN1_READ))
        {
            display_show_number(2);      // Mostrar '2' en display
            leds_shift_right_to_left();   // Efecto LEDs derecha-
izquierda
            l293d_motor_turn_right();     // Motor gira a la derecha
        }

        //11
        if ((BUTTONS_BTN0_READ) && (BUTTONS_BTN1_READ))
        {
            display_show_number('P');    // Mostrar 'P' en display
            leds_blink_all();             // Todos los LEDs parpadeando
            l293d_motor_free_wheel();     // Motor en modo libre
        }
    }
}

```

```

    }
} //fin while
} //fin main

//-----
//-----

// Inicialización de LEDs - Configura pines como salidas
void leds_init(void)
{
    LEDS_DDRX |= (1<<LEDS_LED0); // Configura LED0 como salida
    LEDS_DDRX |= (1<<LEDS_LED1); // Configura LED1 como salida
    LEDS_DDRX |= (1<<LEDS_LED2); // Configura LED2 como salida
    LEDS_DDRX |= (1<<LEDS_LED3); // Configura LED3 como salida
}

// Efecto: Parpadeo de todos los LEDs simultáneamente
void leds_blink_all(void)
{
    // Encender todos los LEDs
    leds_all_on();
    _delay_ms(RETARDO);

    // Apagar todos los LEDs
    leds_all_off();
    _delay_ms(RETARDO);
}

// Efecto: Primero se encienden los LEDs centrales, luego los de las orillas
void leds_center_to_edges(void)
{
    // 1. Encender solo los del centro (LED1 y LED2)
    leds_all_off();
    LEDS_PORTX |= (1 << LEDS_LED1) | (1 << LEDS_LED2);
    _delay_ms(RETARDO);

    // 2. Encender solo las orillas (LED0 y LED3)
    leds_all_off();
    LEDS_PORTX |= (1 << LEDS_LED0) | (1 << LEDS_LED3);
    _delay_ms(RETARDO);
}

// Efecto: Un LED se desplaza de izquierda a derecha
void leds_shift_left_to_right(void)
{
    // Orden de LEDs: LED0 → LED1 → LED2 → LED3 (de izquierda a derecha)
    for(uint8_t i = 0; i < 4; i++)
    {
        leds_all_off(); // Apagar todos
        LEDS_PORTX |= (1 << LEDS[i]); // Encender solo el LED actual
        _delay_ms(RETARDO);
    }
}

// Efecto: Un LED se desplaza de derecha a izquierda
void leds_shift_right_to_left(void)
{

```

```

// Orden de LEDs: LED3 → LED2 → LED1 → LED0 (de derecha a izquierda)
for(int8_t i = 3; i >= 0; i--)
{
    leds_all_off(); // Apagar todos
    LEDS_PORTX |= (1 << LEDS[i]); // Encender solo el LED actual
    _delay_ms(RETARDO);
}

// Encender todos los LEDs simultáneamente
void leds_all_on(void)
{
    LEDS_PORTX |= (1<<LEDS_LED0); // Encender LED0
    LEDS_PORTX |= (1<<LEDS_LED1); // Encender LED1
    LEDS_PORTX |= (1<<LEDS_LED2); // Encender LED2
    LEDS_PORTX |= (1<<LEDS_LED3); // Encender LED3
}

// Apagar todos los LEDs simultáneamente
void leds_all_off(void)
{
    LEDS_PORTX &= ~(1<<LEDS_LED0); // Apagar LED0
    LEDS_PORTX &= ~(1<<LEDS_LED1); // Apagar LED1
    LEDS_PORTX &= ~(1<<LEDS_LED2); // Apagar LED2
    LEDS_PORTX &= ~(1<<LEDS_LED3); // Apagar LED3
}

// Inicialización de botones - Configura pines como entradas con pull-up
void buttons_init(void)
{
    // Botones 0 y 1 con pull-up INTERNO
    BUTTONS_DDRX &= ~(1<<BUTTONS_BTN0); // Configurar como entrada
    BUTTONS_DDRX &= ~(1<<BUTTONS_BTN1); // Configurar como entrada
    BUTTONS_PORTX |= (1<<BUTTONS_BTN0); // Activar pull-up interno
    BUTTONS_PORTX |= (1<<BUTTONS_BTN1); // Activar pull-up interno

    // Botón 2 en PC6 - usar pull-up EXTERNO (RESET del Arduino)
    BUTTONS_DDRX &= ~(1<<BUTTONS_BTN2); // Configurar como entrada
    BUTTONS_PORTX &= ~(1<<BUTTONS_BTN2); // Desactivar pull-up interno
}

// Inicialización del display - Configura todos los segmentos como salidas
void display_init(void)
{
    // Configurar segmentos A-F como salidas en PORTB
    DISPLAY_7SEG_AF_DDRX |= (1<<DISPLAY_7SEG_SEGA);
    DISPLAY_7SEG_AF_DDRX |= (1<<DISPLAY_7SEG_SEGB);
    DISPLAY_7SEG_AF_DDRX |= (1<<DISPLAY_7SEG_SEGC);
    DISPLAY_7SEG_AF_DDRX |= (1<<DISPLAY_7SEG_SEGD);
    DISPLAY_7SEG_AF_DDRX |= (1<<DISPLAY_7SEG_SEGE);
    DISPLAY_7SEG_AF_DDRX |= (1<<DISPLAY_7SEG_SEGF);
    // Configurar segmento G como salida en PORTC
    DISPLAY_7SEG_G_DDRX |= (1<<DISPLAY_7SEG_SEGG);
}

// Muestra caracteres en display 7 segmentos (ánodo común)
void display_show_number(uint8_t number)
{

```

```

switch (number)
{
    case 1: // Mostrar número 1
        DISPLAY_7SEG_AF_PORTX |= (1<<DISPLAY_7SEG_SEGA);
        DISPLAY_7SEG_AF_PORTX &= ~(1<<DISPLAY_7SEG_SEGB);
        DISPLAY_7SEG_AF_PORTX &= ~(1<<DISPLAY_7SEG_SEGC);
        DISPLAY_7SEG_AF_PORTX |= (1<<DISPLAY_7SEG_SEGD);
        DISPLAY_7SEG_AF_PORTX |= (1<<DISPLAY_7SEG_SEGE);
        DISPLAY_7SEG_AF_PORTX |= (1<<DISPLAY_7SEG_SEGF);
        DISPLAY_7SEG_G_PORTX |= (1<<DISPLAY_7SEG_SEGG);
        break;

    case 2: // Mostrar número 2
        DISPLAY_7SEG_AF_PORTX &= ~(1<<DISPLAY_7SEG_SEGA);
        DISPLAY_7SEG_AF_PORTX &= ~(1<<DISPLAY_7SEG_SEGB);
        DISPLAY_7SEG_AF_PORTX |= (1<<DISPLAY_7SEG_SEGC);
        DISPLAY_7SEG_AF_PORTX &= ~(1<<DISPLAY_7SEG_SEGD);
        DISPLAY_7SEG_AF_PORTX &= ~(1<<DISPLAY_7SEG_SEGE);
        DISPLAY_7SEG_AF_PORTX |= (1<<DISPLAY_7SEG_SEGF);
        DISPLAY_7SEG_G_PORTX &= ~(1<<DISPLAY_7SEG_SEGG);
        break;

    case 'E': // Mostrar letra E
        DISPLAY_7SEG_AF_PORTX &= ~(1<<DISPLAY_7SEG_SEGA);
        DISPLAY_7SEG_AF_PORTX |= (1<<DISPLAY_7SEG_SEGB);
        DISPLAY_7SEG_AF_PORTX |= (1<<DISPLAY_7SEG_SEGC);
        DISPLAY_7SEG_AF_PORTX &= ~(1<<DISPLAY_7SEG_SEGD);
        DISPLAY_7SEG_AF_PORTX &= ~(1<<DISPLAY_7SEG_SEGE);
        DISPLAY_7SEG_AF_PORTX &= ~(1<<DISPLAY_7SEG_SEGF);
        DISPLAY_7SEG_G_PORTX &= ~(1<<DISPLAY_7SEG_SEGG);
        break;

    case 'P': // Mostrar letra P
        DISPLAY_7SEG_AF_PORTX &= ~(1<<DISPLAY_7SEG_SEGA);
        DISPLAY_7SEG_AF_PORTX &= ~(1<<DISPLAY_7SEG_SEGB);
        DISPLAY_7SEG_AF_PORTX |= (1<<DISPLAY_7SEG_SEGC);
        DISPLAY_7SEG_AF_PORTX |= (1<<DISPLAY_7SEG_SEGD);
        DISPLAY_7SEG_AF_PORTX &= ~(1<<DISPLAY_7SEG_SEGE);
        DISPLAY_7SEG_AF_PORTX &= ~(1<<DISPLAY_7SEG_SEGF);
        DISPLAY_7SEG_G_PORTX &= ~(1<<DISPLAY_7SEG_SEGG);
        break;

    case 'F': // Mostrar letra F
        DISPLAY_7SEG_AF_PORTX &= ~(1<<DISPLAY_7SEG_SEGA);
        DISPLAY_7SEG_AF_PORTX |= (1<<DISPLAY_7SEG_SEGB);
        DISPLAY_7SEG_AF_PORTX |= (1<<DISPLAY_7SEG_SEGC);
        DISPLAY_7SEG_AF_PORTX |= (1<<DISPLAY_7SEG_SEGD);
        DISPLAY_7SEG_AF_PORTX &= ~(1<<DISPLAY_7SEG_SEGE);
        DISPLAY_7SEG_AF_PORTX &= ~(1<<DISPLAY_7SEG_SEGF);
        DISPLAY_7SEG_G_PORTX &= ~(1<<DISPLAY_7SEG_SEGG);
        break;
}

// Inicialización del controlador L293D - Configura pines como salidas
void l293d_init(void)
{
    L293D_DDRX |= (1<<L293D_EN1); // Enable como salida

```

```

        L293D_DDRX |= (1<<L293D_IN1); // IN1 como salida
        L293D_DDRX |= (1<<L293D_IN2); // IN2 como salida
    }

    // Parada rápida del motor
    void l293d_motor_fast_stop(void)
    {
        L293D_PORTX &= ~(1<<L293D_EN1); // Deshabilitar el motor
        L293D_PORTX &= ~(1<<L293D_IN1); // IN1 = LOW
        L293D_PORTX &= ~(1<<L293D_IN2); // IN2 = LOW → Freno activo
    }

    // Motor en modo libre
    void l293d_motor_free_wheel(void)
    {
        L293D_PORTX &= ~(1<<L293D_IN1); // IN1 = LOW
        L293D_PORTX &= ~(1<<L293D_IN2); // IN2 = LOW
        L293D_PORTX &= ~(1<<L293D_EN1); // Apagar enable
    }

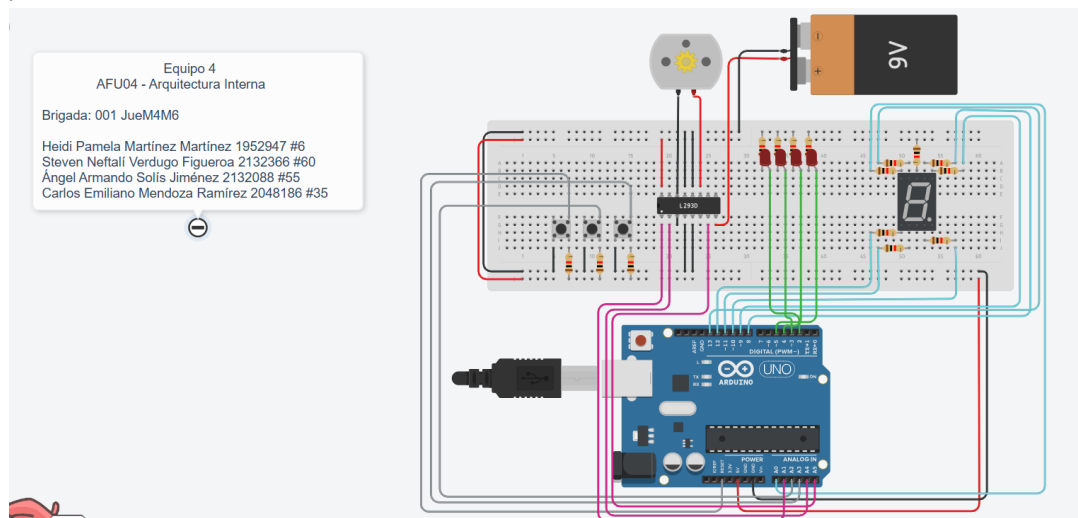
    // Motor gira en sentido DERECHA
    void l293d_motor_turn_right(void)
    {
        L293D_PORTX |= (1<<L293D_EN1); // Habilitar el motor
        L293D_PORTX |= (1<<L293D_IN1); // IN1 = HIGH
        L293D_PORTX &= ~(1<<L293D_IN2); // IN2 = LOW
    }

    // Motor gira en sentido IZQUIERDA
    void l293d_motor_turn_left(void)
    {
        L293D_PORTX |= (1<<L293D_EN1); // Habilita el motor
        L293D_PORTX &= ~(1<<L293D_IN1); // IN1 = LOW
        L293D_PORTX |= (1<<L293D_IN2); // IN2 = HIGH
    }
}

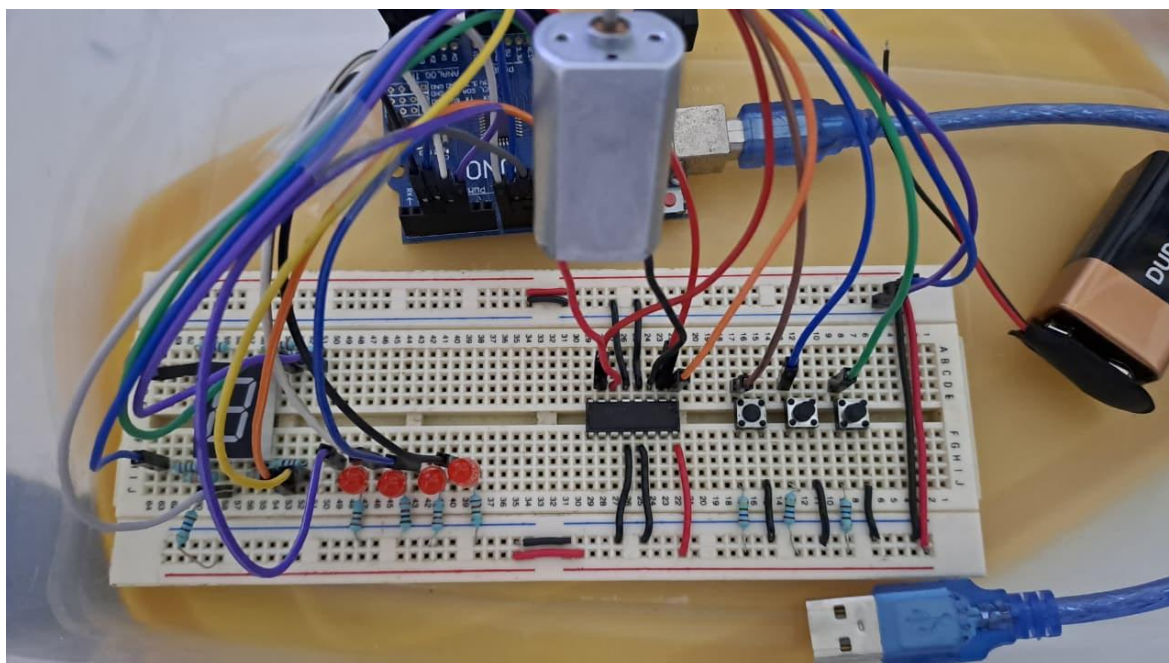
```

7. FOTOGRAFÍA DEL PROTOTIPO

ESQUEMÁTICO EN TINKERCAD



PROTOTIPO EN FÍSICO



CONCLUSIÓN PERSONAL

Heidi Pamela Martínez Martínez 1952947 #6

Aprendí que la teoría de las tablas de verdad y máquinas de estado cobra significado cuando enfrentas problemas prácticos como motores que giran al revés o displays que muestran solo puntos decimales; lo más difícil fue lidiar con la frustración inicial de conexiones mal hechas y lógica invertida en los puertos, pero superarlo me enseñó que en sistemas embebidos la paciencia y revisar código, conexiones y hardware son tan cruciales como el conocimiento técnico.

Steven Neftalí Verdugo Figueroa 2132366 #60

La implementación del sistema electrónico basado en un microcontrolador permitió comprender de manera práctica el control de dispositivos externos mediante señales digitales. Durante el desarrollo se aplicaron conceptos de electrónica digital, programación en lenguaje C y diseño esquemático con software EDA, reforzando la relación entre hardware y software en sistemas embebidos. La simulación y posterior montaje del prototipo facilitaron la verificación del funcionamiento, así como el análisis de posibles ajustes en código y conexiones.

En conjunto, esta actividad contribuyó al fortalecimiento de nuestras habilidades en diseño, programación y diagnóstico de sistemas de control electrónico, mejorando nuestra comprensión del funcionamiento interno de los microcontroladores y su aplicación en proyectos de automatización.

Ángel Armando Solís Jiménez 2132088 #55

La realización de esta actividad me permitió aplicar los conocimientos adquiridos sobre microcontroladores y sistemas de control, integrando teoría y práctica en el diseño de un sistema electrónico capaz de controlar un motor de corriente directa mediante el microcontrolador ATMEGA328P y el driver L293D, así como la gestión de LEDs y un display de 7 segmentos para mostrar el estado del sistema.

A lo largo del proyecto, seguí el procedimiento propuesto, comenzando por el análisis preliminar y el diseño conceptual, lo cual me ayudó a comprender los requisitos y las posibilidades de la solución. El desarrollo del diagrama de bloques y esquemático me permitió visualizar la interconexión de los componentes y asegurarme de cumplir con las especificaciones. La codificación y simulación fueron etapas clave para verificar el correcto funcionamiento antes de construir el prototipo físico. Durante la implementación, la documentación detallada y la fotografía del prototipo facilitaron la presentación y el reporte del trabajo realizado.

Carlos Emiliano Mendoza Ramírez 2048186 #35

Este proyecto me permitió comprender de forma integral cómo se desarrollan los sistemas electrónicos de control, combinando tanto la parte teórica como la práctica. Aprendí que detrás de un proyecto funcional existe una planeación detallada que incluye la elaboración de diagramas de bloques, diagramas de flujo y la correcta selección de componentes electrónicos. Estos elementos son fundamentales para entender el funcionamiento del circuito y anticipar posibles fallos o mejoras. También aprendí la importancia de la programación en microcontroladores como herramienta para automatizar procesos y controlar dispositivos, en este caso un motor, junto con otros periféricos como LEDs y displays. Pude ver cómo el hardware y el software deben complementarse para lograr un sistema eficiente, estable y seguro.

REFERENCIAS BIBLIOGRÁFICAS

Mauricio, O. (2020). Display 7 segmentos ánodo común | CodigoElectronica. Codigoelectronica.com. <http://codigoelectronica.com/blog/display-7-segmentos-anodo-comun>

Stallings, W. (2017). Computer Organization and Architecture. Pearson.

Mazidi, M., Naimi, S., & Naimi, S. (2018). 8051 Microcontroller and Embedded Systems. Pearson.

Kernighan, B., & Ritchie, D. (1988). The C Programming Language. Prentice Hall.

Maxfield, C. (2004). The Design Warrior's Guide to FPGAs. Elsevier.

Kip Irvine, R. (2015). Assembly Language for x86 Processors. Pearson.

Rabey, J. (2009). Digital Integrated Circuits. Prentice Hall.

Arduino. (2025). Arduino UNO Rev3 with Long Pins. Arduino.cc. <https://docs.arduino.cc/retired/boards/arduino-uno-rev3-with-long-pins/>